VINUNIVERSITY

Monitoring, Logging & Observability

AICB-P1 · Ngày 13 · Biết agent đang chạy thế nào trước khi user phàn nàn

Tên Giảng Viên

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

“Agent bạn deploy hôm Day 12 chạy ngon. 3 ngày sau: latency tăng gấp đôi, cost tăng 300 phần trăm, và 1 trên 20 câu trả lời là bịa. Bạn biết những điều này khi nào? — Khi user phàn nàn. Đó là cách tệ nhất, và đắt nhất, để phát hiện vấn đề.”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Vì sao agent cần observability
2. 3 pillars + pillar thứ 4
3. AI-specific metrics
4. Structured logging
5. Distributed tracing cho agent
6. Production stack: Prometheus + Grafana
7. Dashboard design
8. Alerting
9. Debug 1 incident bằng trace
10. Human feedback & online eval
11. Privacy & compliance khi logging
12. Checklist, Lab 13 & tổng kết

Mục Tiêu Ngày 13 (Bloom's Taxonomy)

- **Remember** — liệt kê **3 pillars** (metrics, logs, traces) + **pillar thứ 4** (continuous eval) và 6 nhóm AI-specific metrics
- **Understand** — giải thích vì sao P99 quan trọng hơn average, và alert theo symptom khác theo cause thế nào
- **Apply** — implement **structured logging** (JSON + correlation ID + PII redaction) cho agent đang deploy
- **Analyze** — đọc trace waterfall và xác định bottleneck trong agent pipeline nhiều bước
- **Evaluate** — so sánh chiến lược sampling (đầu vs cuối request) cho trace, và alert nào nên page vs chỉ nên tạo ticket
- **Create** — thiết kế monitoring blueprint với alert rules (symptom-based) và dashboard 3 layers

Agent có observability đầy đủ

Bạn biết nó chạy thế nào mà **không cần hỏi user**.

- Structured logging pipeline: JSON, correlation ID, input/output đã redact PII
- Tracing: Langfuse (hoặc backend zero-key) connected, ≥ 10 traces
- Dashboard: latency P50/95/99 + TTFT, cost/ngày, error rate, token usage, tool-call success
- ≥ 3 alert rules $\rightarrow$ Slack (symptom-based); 1 daily cost budget alert; 1 incident note đọc từ trace

Vì Sao Agent Cần Observabil-ity

“It works” không đủ cho production — cần biết nó chạy TỐT đến đâu, chậm ở đâu, tốn bao nhiêu, và khi nào sắp hỏng

01

Từ Artifact Day 12 Sang Production Reality

Day 12 đã làm được

- Agent deployed trên cloud
- Public URL hoạt động
- Health check endpoint
- Basic authentication

Nhưng chưa trả lời được

- Agent đang chậm hay nhanh?
- Tốn bao nhiêu tiền mỗi ngày?
- Bao nhiêu request fail (hoặc trả lời sai)?
- Khi nào cần scale up?

Lưu ý: Không có monitoring, bạn chỉ biết agent hỏng **khi user phàn nàn**. Health check "200 OK" không có nghĩa câu trả lời đúng.

MINI QUEST 1

Câu hỏi: Với agent bạn đang xây (Day 12), bạn đang dùng hệ thống monitoring/logging nào — và dùng như thế nào?

Thảo luận nhóm nhỏ (3–4 người), sau đó chia sẻ nhanh với cả lớp:

- Bạn có đang log gì không? Ở đâu — console, file, hay một service riêng?
- Bạn có biết agent tốn bao nhiêu request/ngày, cost bao nhiêu không?
- Nếu agent lỗi hoặc trả lời sai, bạn phát hiện bằng cách nào: tự thấy hay chờ user báo?
- Có ai từng gặp bug mà không tìm được nguyên nhân vì thiếu log/trace chưa?

Thời gian: 15 phút (10' thảo luận nhóm + 5' chia sẻ cả lớp)

Monitoring vs Observability — Hai Khái Niệm Khác Nhau

VINUNIVERSITY

Monitoring

Theo dõi các câu hỏi **đã biết trước**.

- Dashboard + alert dựng sẵn
- Trả lời: “X có hỏng không?”
- Tốt cho failure mode đã lường trước
- “Known-knowns”

Observability

Thuộc tính của hệ thống: hỏi câu **mới** mà không cần deploy code.

- Telemetry đủ giàu (metrics + logs + traces)
- Trả lời: “TẠI SAO X hỏng?”
- Tốt cho failure mode **chưa từng gặp**
- “Unknown-unknowns”

Observability AI Khác Gì Monitoring Phần Mềm Truyền Thống?

Không test bằng so sánh string

Cùng input, output khác nhau mỗi lần. Phải đo **chất lượng**, không chỉ pass/fail.

App không "crash"

Nó vẫn trả 200 OK nhưng câu trả lời tệ dần. Không có exception để bắt.

Cost tính theo token

Mỗi request tốn tiền theo số token. Một bug loop có thể đốt budget trong vài giờ — CPU/RAM không nói cho bạn biết.

Failure mode riêng của AI

Hallucinated tool args, vòng lặp vô tận, context overflow, prompt injection — những lỗi mà APM truyền thống không có khái niệm.

Cost of Not Monitoring

Silent failures

Agent trả lời sai nhưng không ai biết.
User mất niềm tin dần dần.
Đến khi phát hiện thì đã mất user.

Surprise bills

Token cost tăng dần mà không alert.
Cuối tháng nhận bill gấp 5 lần dự kiến.
Đốt hết budget trước khi kịp react.

Gradual degradation

Latency P95 tăng 10ms mỗi tuần.
6 tuần sau: chậm gấp đôi.
Không ai để ý vì không có baseline.

Debugging nightmare

Bug report: “agent sai hôm qua.”
Không log, không trace.
Không reproduce được. Không fix được.

Observability: Vài Cột Mốc



Logs (text)

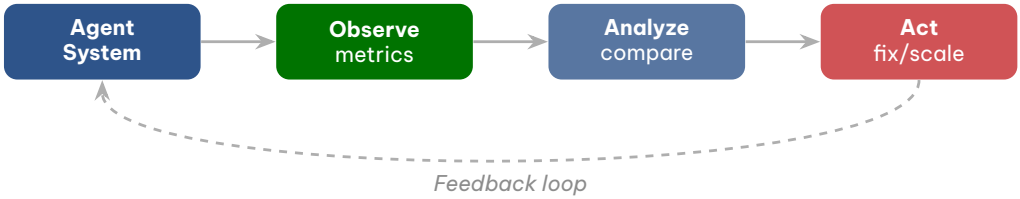
Metrics & Prometheus 2012

Grafana 2014

Tracing & OTel 2019

LLM-native 2023+

Control Theory — Observability Là Một Feedback Loop



Mean Time To Detect

Từ khi sự cố xảy ra đến khi phát hiện.

Mean Time To Recover

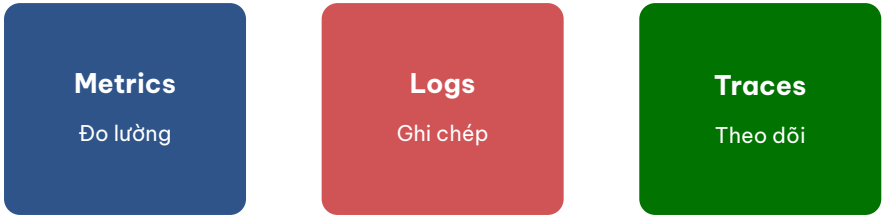
Từ khi phát hiện đến khi fix xong.

3 Pillars + Pillar Thứ 4

Metrics nói bao nhiêu / bao lâu, logs nói chuyện gì xảy ra, traces nói tại sao — và với AI có pillar thứ 4: câu trả lời có còn ĐÚNG không

02

3 Pillars of Observability



Logs nói chuyện gì xảy ra. **Metrics** nói bao nhiêu và bao lâu. **Traces** nói tại sao.

Pillar thứ 4

Với AI system, ba pillar truyền thống không trả lời được câu hỏi quan trọng nhất: **câu trả lời có còn đúng không?** Pillar thứ 4 = đo chất lượng output liên tục trên production.

- HTTP 200 khác correctness — request "thành công" vẫn có thể là câu trả lời bịa
- Latency thấp khác hữu ích — trả lời nhanh nhưng sai còn tệ hơn
- Error rate 0% khác không đốt tiền — chi phí vẫn có thể tăng vọt

Day 13 vs Day 14

Day 13 đo chất lượng **liên tục trên production** (online). Day 14 đo chất lượng **có hệ thống bằng benchmark** (offline). Cả hai bổ sung cho nhau.

Tại Sao Chỉ Logs Là Không Đủ?

Chỉ có logs

- biết request nào fail
- nhưng không biết fail rate là bao nhiêu
- không biết latency đang tăng dần
- không biết bottleneck ở đâu

Đủ pillars

- metrics cho biết trend (tăng/giảm)
- logs cho biết chi tiết từng request
- traces cho biết chậm ở bước nào
- eval cho biết chất lượng còn tốt không

Pillar Nào Trả Lời Câu Hỏi Nào?

Câu hỏi	Pillar	Công cụ ví dụ
“Error rate có tăng không?”	Metrics	Prometheus, Grafana
“Request req-abc đã làm gì?”	Logs	Loki, JSON logs
“Chậm ở bước nào trong agent loop?”	Traces	Langfuse, Tempo
“Câu trả lời còn đúng không?”	Eval (4th)	LLM-judge, RAGAS

Nguyên tắc

Chọn pillar theo câu hỏi bạn cần trả lời. Đừng thu thập telemetry chỉ vì có thể — mỗi data point đều tốn tiền lưu trữ (xem §9).

AI-Specific Metrics

03

Monitoring truyền thống đo CPU, RAM, uptime — AI agent cần thêm: token, cost, TTFT, chất lượng, tool-call success, retrieval quality

4 Nhóm Metrics Cho AI Agent

Performance

- Latency P50 / P95 / P99
- Time to first token (TTFT)
- Throughput (req/s, tokens/s)
- LLM call duration

Cost

- Tokens per request (in/out)
- Cost per request / per task
- Cost per day/user/feature
- Cache hit rate

Quality (pillar 4)

- Hallucination / faithfulness
- Task completion rate
- Thumbs up/down rate
- Guardrail trigger rate

Reliability

- Error rate, uptime
- Tool-call success rate
- Retry rate, loop rate
- Retrieval recall/empty-result

4 Golden Signals + 2 Cho AI Agent

Google SRE — 4 Golden Signals

1. **Latency** — thời gian phản hồi
2. **Traffic** — request rate (QPS)
3. **Errors** — error rate
4. **Saturation** — tài nguyên còn bao nhiêu

AI agent cần thêm 2

5. **Cost** — $/request, $/user, token usage
6. **Quality** — hallucination rate, CSAT, groundedness

RED vs USE — Hai Phương Pháp Observability

RED (request-centric)

- **R**ate — requests/giây
- **E**rrors — error rate
- **D**uration — latency P50/P95/P99

Góc nhìn user: tôi gửi request, được gì?

USE (resource-centric)

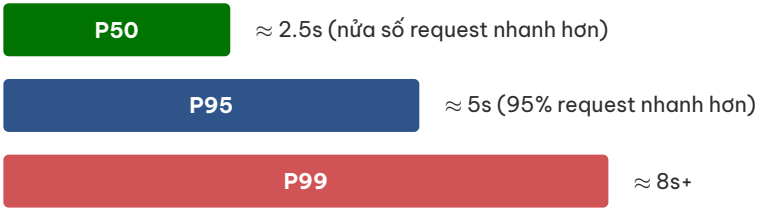
- **U**tilization — tài nguyên dùng %
- **S**aturation — có queue/chờ không?
- **E**rrors — lỗi của resource

Góc nhìn resource: LLM API, queue đang làm gì?

Kết hợp cả hai

Agent chậm (RED: Duration P95 tăng) → debug bằng USE (LLM rate-limit utilization 95%) → bị throttle → upgrade tier hoặc fallback.

Latency: Percentiles + TTFT



TTFT (Time To First Token)

Thời gian từ lúc gửi request đến token đầu tiên. Quyết định cảm giác "nhanh". Điển hình 2026: P50 ≈ 0.5–1.0s, P95 ≈ 1.5–2.5s.

Lưu ý: Trung bình (average) ẩn long tail. P95 mới là trải nghiệm thật. **Reasoning mode** là lớp latency riêng (chậm hơn 5–30x) — tách ra khi đo.

Percentile Math — Vì Sao P99 Quan Trọng

Bài toán — Agent có P99 = 5s, user chat 10 lượt. Xác suất gặp ≥ 1 lần > 5s:

$$P = 1 - 0.99^{10} \approx 9.6\%$$

- 1/10 user sẽ gặp lag rất tệ
- 1.000 user/ngày $\rightarrow$ 96 user bức xúc
- Họ là người tweet negative, churn

Amazon/Google/Meta

"Every 100ms of latency cost 1% of sales." $\rightarrow$ optimize tail, không chỉ average. P99/P99.9 là KPI chính thức.

Lưu ý: Tail latency **compounds** trong agentic workflow nhiều bước — 5 bước, mỗi bước có P99 riêng $\rightarrow$ gần như chắc chắn 1 bước chạm tail. Đo P99 cho **cả pipeline**, không chỉ từng call.

Token & Cost Metrics — Output Đắt Hơn Input

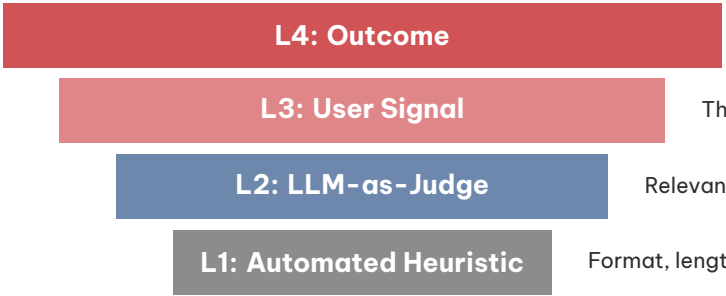
Model (2026)	Input $/1M	Output $/1M	Tỉ lệ out:in
Claude Haiku 4.5	1	5	5x
Claude Sonnet 4.6	3	15	5x
Claude Opus 4.8	5	25	5x
OpenAI GPT-5.5	5	30	6x
Gemini 3.1 Pro	2	12	6x

cost-per-task $\neq$ cost-per-LLM-call

1 task của agent có thể gọi LLM nhiều lần (plan + tool + synthesize). Đo cost theo **task** và rollup theo ngày/user/feature, không chỉ theo từng call.

Lưu ý: Output token đắt 5–6x input. Một agent "nói nhiều" tốn tiền hơn nhiều so với độ dài prompt gợi ý $\Rightarrow$ dashboard token phải **tách input vs output**.

Quality Metrics — Kim Tự Tháp 4 Tầng



Cân bằng cả 4

L1 rẻ, realtime nhưng không nói được quality thực. L4 là ground truth nhưng lag hàng tuần. Production cần cả 4: L1/L2 để alert, L3/L4 để confirm trend.

Hallucination — Phát Hiện Thế Nào?

Hallucination

Agent trả lời "rất tự tin" nhưng sai sự thật. Không có 1 metric duy nhất → cần combo 4 patterns.

Mỗi claim trong output → check có trong retrieved context không. Tool: RAGAS faithfulness, TruLens. Gọi LLM 3 lần (temp 0.7); 3 câu mâu thuẫn → nghi ngờ. Cost 3x → chỉ sample 1%.

Extract entities (tên, số, dates) → cross-check DB/API. Cho finance, medical. "Was this helpful?" + regenerate click = tín hiệu hallucination. Chậm nhưng rẻ và thật.

Lưu ý: Air Canada (2024): chatbot bịa chính sách bereavement fare. Nếu có **groundedness check** với policy DB → block từ đầu, tránh kiện tụng.

Quality Metrics — Đo Cái Khó Đo

Tín hiệu trực tiếp

- Hallucination rate (bịa thông tin)
- Faithfulness / groundedness (bám nguồn)
- Task-completion rate

Tín hiệu gián tiếp (từ user)

- Thumbs up/down
- Regenerate/rephrase rate
- Abandon/escalate-to-human rate

Cách làm

Không thể chấm tay mọi request. Sample 1% → chấm bằng LLM-as-judge/RAGAS → đẩy thành 1 metric (gauge) → alert khi tụt. Chi tiết về eval hệ thống: Day 14.

Tool-Call & Retrieval Metrics

VINUNIVERSITY

Tool calls

- Success rate / schema-fail rate
- Timeout rate
- Loop rate (gọi lặp lại)
- Args hallucination (bịa tham số)

Retrieval (RAG)

- Recall@k (proxy)
- Empty-result rate
- Chunk relevance
- Retrieval latency

Failure mode riêng của agent

Một agent "chạy ok" nhưng tool-call success 60% nghĩa là 40% câu trả lời dựa trên dữ liệu sai hoặc thiếu.

Reliability — Error Taxonomy

Loại lỗi	Nguyên nhân	Cách handle
LLM API 5xx	Provider down / rate limit	Retry (exp. backoff), fallback model
LLM timeout	Slow provider, network	Circuit breaker, client timeout $<$ server
Tool call failed	External API lỗi	Retry, graceful degradation
Tool schema invalid	LLM sinh JSON lỗi	Re-prompt với error feedback
Guardrail block	Content policy vi phạm	Log + user-friendly message
Empty response	LLM refuse / filter	Alternate prompt, escalate to human
Context overflow	Input $>$ limit	Truncate, summarize history

Lưu ý: Track `error_type` trong log. Alert fire $\rightarrow$ biết ngay gọi ai: LLM provider? tool owner? prompt engineer? "Error rate 5%" không taxonomy = không có gì để fix.

Drift — Khi Data/Model Thay Đổi Âm Thầm

3 loại drift cần monitor

- **Data drift** — input distribution đổi (user hỏi kiểu mới)
- **Concept drift** — mapping input→output đổi (luật mới)
- **Model drift** — provider update model, behavior đổi

Phát hiện: PSI, KL-divergence, embedding drift (cosine).

$$PSI = \sum_i (p_i - q_i) \ln \frac{p_i}{q_i}$$

< 0.1 stable · 0.1–0.25 mild · > 0.25 significant (cần retrain).

Lưu ý: 2024: OpenAI silently update GPT-4 → format output đổi → nhiều pipeline breaks âm thầm. Không drift monitoring = không biết cho đến khi user bỏ đi.

Metric Nào Cho Ai?

Stakeholder	Quan tâm	Metrics
Engineering	System health, debug	Latency P95, error rate, tool-call failure
Product	User experience	Satisfaction, task completion, hallucination rate
Finance / Ops	Cost control	Cost/ngày, tokens/request, cost by model
Leadership	ROI overview	Adoption, cost vs value, uptime

Nguyên tắc

Dashboard cho stakeholder phải nói bằng **ngôn ngữ business**, không phải ngôn ngữ kỹ thuật.

Structured Logging

04

Log không cấu trúc giống ghi chú tay — khó search, khó aggregate. Structured logging biến log thành DATA query được

Unstructured vs Structured Log

```
# Unstructured: kho search / filter / aggregate
# 10:23:45 INFO Agent responded  10:23:46 ERROR Tool failed
# Structured JSON: query/aggregate/correlate duoc
log = {
    "ts": "2026-03-18T10:23:45Z", "level": "INFO",
    "correlation_id": "req-abc123", "event": "agent_response",
    "latency_ms": 1250, "input_tokens": 640, "output_tokens": 250,
    "cost_usd": 0.0057, "model": "claude-sonnet-4-6",
}
```

Lợi ích

Query được như data: **filter theo field**, **aggregate**, **correlate across services** — điều text log không làm được.

Log Gì Cho 1 LLM Call?

- ☑ correlation_id (nối mọi log của 1 request)
- ☑ model + version, provider
- ☑ prompt template id (KHÔNG log raw prompt chứa PII)
- ☑ input_tokens / output_tokens, latency_ms, TTFT
- ☑ tool calls + kết quả (đã sanitize), finish_reason
- ☑ cost_usd (tính từ token)
- ☑ eval score (nếu có), error + stack trace

Log Gì Và Không Log Gì?

Nên log

- Input (đã sanitize)
- Output summary
- Tool calls + results
- Latency, tokens, cost
- Errors + stack traces
- Correlation ID

KHÔNG log

- PII (tên, SĐT, CCCD, email)
- Full prompts chứa sensitive data
- API keys, tokens, secrets
- Raw user data chưa sanitize
- Quá nhiều DEBUG ở production

PII Redaction Trong Thực Tế

Kỹ thuật

- Regex: email, SĐT, thẻ, CCCD
- NER / entity detection: tên người, địa chỉ (Microsoft Presidio)
- Hashing / tokenization: giữ tính duy nhất, bỏ giá trị gốc
- Allowlist: chỉ log field đã duyệt

Presidio (OSS/MIT)

Phát hiện 50+ loại PII, redact/mask/hash qua "operators". Hỗ trợ tiếng Việt yếu — cần custom recognizer cho CCCD/SĐT VN.

Lưu ý: Log PII = vi phạm PDPL (Việt Nam) / GDPR. **Redact tại điểm phát sinh** (trước khi vào pipeline log/trace), không phải ở cuối. Nối với guardrails Day 11.

Log Levels Đúng Cách

Level	Khi nào dùng	Ví dụ
`DEBUG`	Dev only, rất chi tiết	Full prompt, intermediate state
`INFO`	Normal flow, milestone	Request received, response sent
`WARN`	Degraded nhưng vẫn chạy	Retry succeeded, fallback used
`ERROR`	Failed, cần attention	Tool timeout, LLM error

Rule of thumb

Production chạy **INFO level**. Khi debug issue cụ thể, **tạm bật DEBUG** cho 1 request ID, xong tắt lại.

Correlation ID — Nối Tất Cả Lại

```python
import uuid

def handle_request(user_input):
    req_id = str(uuid.uuid4())[:8]  # 1 id cho toan bo request

    log.info("request_received",
             correlation_id=req_id,
             input_length=len(user_input))

    result = agent.run(user_input, req_id=req_id)

    log.info("response_sent",
             correlation_id=req_id,
             latency_ms=result.latency,
             output_tokens=result.output_tokens)
    return result
```

Correlation ID nối mọi log entry của 1 request, dù đi qua nhiều service. Nó cũng là mầm của `trace_id` — cầu nối sang distributed tracing (§5).

structlog + contextvars — Correlation ID Tự Động

```python
import structlog, uuid
from structlog.contextvars import bind_contextvars, clear_contextvars

structlog.configure(processors=[
    structlog.contextvars.merge_contextvars,   # tu dong chen context
    structlog.processors.add_log_level,
    structlog.processors.TimeStamper(fmt="iso", utc=True),
    structlog.processors.JSONRenderer(),        # -> JSON moi dong
])
log = structlog.get_logger()

async def handle_request(req):
    clear_contextvars()
    bind_contextvars(correlation_id=str(uuid.uuid4())[:8],
                     user_id=req.user_id, feature=req.feature)
    # Tu day: moi log.* tu dong co 3 fields tren
    log.info("request_received", query_len=len(req.query))
    return await agent.run(req.query)
```

Log Sampling — Khi Volume Quá Cao

Bài toán — 100k req/ngày $\times$ 10 log/req = 1M entries/ngày. Datadog $\sim$\$0.10/1k $\rightarrow$ \$100/ngày chỉ riêng log. Không scalable.

Strategies

- **Head** — quyết định ngay đầu trace (rẻ, có thể miss errors)
- **Tail** — quyết định sau khi xong (đắt, giữ 100% errors)
- **Reservoir** — giữ N mẫu uniform

100% ERROR + WARN · 10% INFO · 1% DEBUG · 100% request $>$10s (tail-on-latency) · 100% cost $>$\$1/req (outliers)

Lưu ý: Sampling giảm cost 10–100x nhưng mất visibility vào normal pattern. Giữ **100% errors** là non-negotiable — đó là data debug quan trọng nhất.

Log Aggregation Stacks

Stack	Khi nào dùng	Cost/tier
ELK	Full-text search mạnh, complex queries	Tự host, OSS
Loki	Label-based (giống Prometheus), rẻ	Tự host, OSS
Datadog Logs	Setup nhanh, alert tốt, đắt ở scale	SaaS ~$0.10/GB
CloudWatch	Đã ở AWS, tích hợp IAM	~$0.50/GB ingest
BigQuery	Analytics SQL, long retention	~$0.02/GB scan

Cho lab

Langfuse tự là log store cho LLM call (free tier). Dev local: `stdout` JSON + `jq` là đủ. Scale up mới cần ELK/Loki — đừng dựng cluster Elasticsearch cho MVP.

Log Stacks — Ai Làm Cái Gì



Elastic (ELK)



Loki



Datadog



CloudWatch



BigQuery

Audit Log — Tách Biệt Với App Log

VINUNIVERSITY

Audit log

Record "who did what when" cho compliance, legal, security — khác hẳn app log dùng để debug.

App log

- Mục đích: debug, performance
- Retention: 30–90 ngày
- Có thể sample/sửa/xóa
- Truy cập: dev team

Audit log

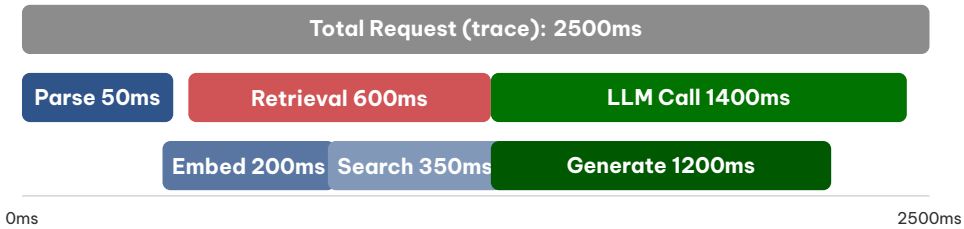
- Mục đích: compliance, forensics
- Retention: 2–7 năm (tùy ngành)
- Không sample; append-only
- Truy cập: restricted (compliance)

05

Distributed Tracing Cho Agent

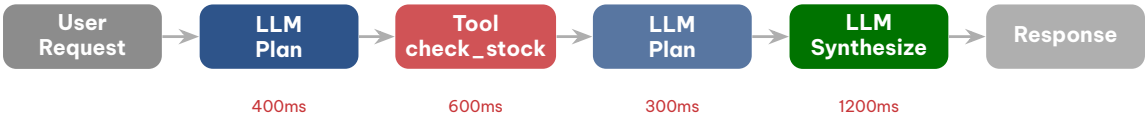
Log cho biết gì xảy ra ở từng bước; trace cho biết hành trình của 1 request qua LLM → tool → LLM và mất bao lâu ở mỗi bước

Trace, Span, Parent–Child



Mỗi hàng ngang là 1 **span**. Tất cả spans của 1 request tạo thành 1 **trace**. Span con lồng trong span cha. Nhìn trace biết ngay **bottleneck ở đâu**.

Trace Cho Agent Pipeline (Multi-Step Loop)



Insight từ trace này

Agent loop = chuỗi LLM $\leftrightarrow$ tool. Trace cho thấy mỗi vòng tốn bao lâu. Ở đây 2 LLM call chiếm 64% latency $\Rightarrow$ tối ưu prompt/model trước.

OpenTelemetry (OTel)

Chuẩn mở để sinh và xuất telemetry (traces, metrics, logs). Instrument code **một lần** bằng OTel → gửi tới bất kỳ backend nào (đổi backend không sửa code).



Lưu ý: Tránh vendor lock-in. Cùng 1 trace có thể vào Langfuse (UI cho LLM) và Tempo (lưu trữ rẻ) song song.

Đọc 1 Trace Multi-Step

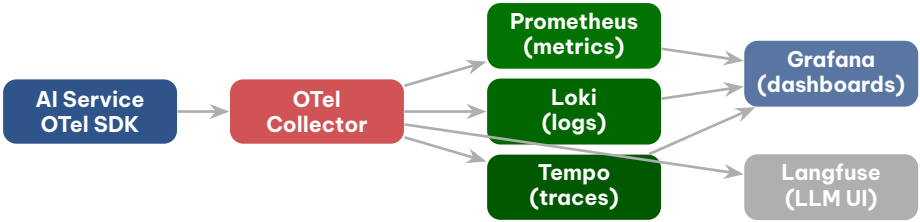
```
# Span tree cua 1 agent run (ten span = "{operation} {model/tool}")
invoke_agent ecommerce-agent                          2500ms
|- chat claude-sonnet-4-6         (plan)                400ms
|- execute_tool check_stock                               600ms  <-- cham!
|- chat claude-sonnet-4-6         (plan)                300ms
`- chat claude-sonnet-4-6         (synthesize)          1200ms
# Doc: tong 2500ms; check_stock 600ms la I/O cham,
# 2 lan synthesize chiem 1600ms -> toi uu prompt/model truoc.
```

Đọc trace = đọc cây span: bước nào lâu nhất, bước nào lỗi, bước nào lặp. Đây là kỹ năng dùng lại ở §10 (debug incident).

Production Stack: Prometheus + Grafana

Prometheus thu metrics, Grafana vẽ dashboard, OTel Collector kết nối tất cả — bộ stack open-source kinh điển, nay có thêm lớp LLM

Kiến Trúc Stack — Tổng Quan



- Instrument 1 lần (OTel) → Collector fan-out 3 backend chuyên biệt; Grafana vẽ tất cả, Langfuse nhận trace LLM song song
- Prometheus: **Counter**/**Gauge**/**Histogram** (phân phối → P95/P99); dashboard nên lưu JSON/YAML trong git (Dashboard-as-Code), review qua PR

Lưu ý: **Cardinality** là kẻ đốt tiền thầm lặng: label `user_id`/raw prompt → bùng nổ time-series — Coinbase từng nhận hóa đơn Datadog $65 triệu (2022) phần lớn vì việc này. Chỉ dùng label thấp-cardinality (`model`, `status`).

Công Cụ Thực Tế Trong Stack Này



OpenTelemetry



Prometheus



Grafana

loki

Loki

tempo

Tempo

langfuse

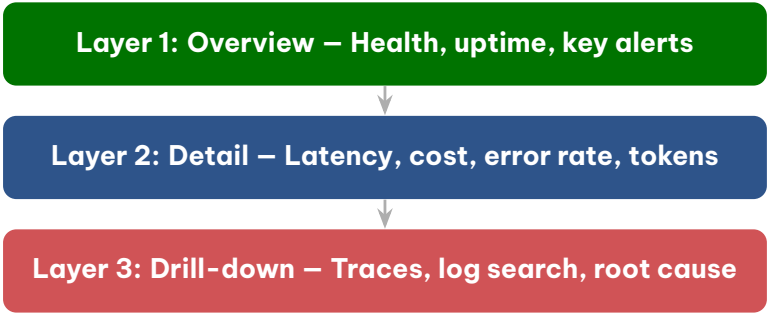
Langfuse

Dashboard Design

07

Mỗi stakeholder một câu hỏi, một dashboard — nhồi mọi thứ vào một màn hình là cách chắc chắn không ai nhìn

Dashboard — 3 Layers



Nguyên tắc

Mỗi stakeholder chỉ cần nhìn 1 layer. Leadership cần overview, không cần trace. Engineer cần drill-down, không cần revenue chart.

Dashboard Anti-patterns — 5 Điều Nên Tránh

1. **“Wall of metrics”** — 30 panel, không ai nhìn hết. Giới hạn 6–8 panel/layer.
2. **Time range mặc định quá dài** — default 1 giờ cho ops, không phải 1 tháng (che mất spike).
3. **Không có baseline/threshold line** — P95 = 2.1s tốt hay xấu? Luôn vẽ đường SLO lên chart.
4. **Metric không có đơn vị/context** — “Cost: 1250” là gì? USD? ngày? Luôn label đầy đủ.
5. **Không auto-refresh** — ops dashboard cần realtime (15–30s); monthly report thì khác.

> **Lưu ý:** Đưa dashboard cho người ngoài team xem 30s → họ nói được “hệ thống OK” hay “có vấn đề ở X” không? Nếu không, redesign.

Alerting

Metrics chỉ có giá trị nếu có người nhìn. Alert sai cách còn tệ hơn không có — biết khi nào cần page ai, khi nào chỉ cần ticket

Alert Rules Cho AI Agent

Metric	Threshold	Severity	Channel
Latency P95	$>$ 5 giây	Warning	Slack
Error rate	$>$ 5%	Critical	Slack + Email
Daily cost	$>$ budget ngày	Critical	Email + SMS
Tool-call failure	$>$ 10%	Warning	Slack
Eval score	tụt $>$ 10%	Warning	Slack
Uptime	$<$ 99%	Critical	PagerDuty

Nguyên tắc

Alert phải **actionable**. Nếu nhận alert mà không biết làm gì, alert đó cần redesign hoặc bỏ.

Symptom-Based vs Cause-Based Alerting

VINUNIVERSITY

Symptom-based (NÊN page)

Alert trên cái **user cảm nhận được**.

- Error rate / latency vượt ngưỡng
- "Câu trả lời sai tăng vọt"
- Ít false positive, luôn thật

Bốn golden signals: Latency, Traffic, Errors, Saturation.

Cause-based (để DEBUG)

Alert trên **nguyên nhân có thể**.

- "CPU 80%", "cache miss cao"
- Có thể chưa ảnh hưởng user
- Nhiều noise nếu để page

Dùng để chẩn đoán, không phải để gọi người.

Alert Fatigue — Khi Alert Quá Nhiều

VINUNIVERSITY

Alert fatigue xảy ra khi

- quá nhiều alert không quan trọng
- mọi người bắt đầu ignore
- alert thật bị lẫn trong noise
- team mất tin tưởng hệ thống

Cách tránh (Google SRE)

- chỉ page khi cần **hành động ngay**
- mỗi page phải đòi **trí tuệ** (không robotic)
- page về vấn đề **mới**, chưa từng thấy
- phần còn lại → ticket / dashboard

Lưu ý: Nếu team ignore alert thường xuyên, hệ thống alerting đang **tệ hơn không có**. Nguy hiểm nhất: 1 page thật bị che lấp trong noise.

Severity & escalation

- SEV1 (down/critical) → page ngay
- SEV2 (degraded) → Slack, giờ làm
- SEV3 (minor) → ticket
- Escalation: primary → secondary → lead

MTTD / MTTR

MTTD = thời gian phát hiện. MTTR = thời gian khắc phục. Mục tiêu observability: giảm cả hai.

Lưu ý: Bối cảnh VN: lịch on-call theo UTC+7; tránh deploy lớn dịp Tết; nhớ nghĩa vụ báo cáo sự cố dữ liệu **72 giờ** theo PDPL (§12).

Template cho mỗi alert:

- Title rõ: “[P1] Agent P95 latency > 5s cho feature=summary”
- Severity: P1 (page ngay) / P2 (giờ hành chính) / P3 (ticket)
- Impact: “5% user đang bị chậm > 5s”
- Current value: “P95 = 6.3s, bình thường 1.8s, threshold 5s”
- Dashboard link (pre-filtered) + Trace link (top 10 chậm nhất)
- Runbook link (playbook fix) + On-call owner

Lưu ý: Alert không có **runbook** = alert không thể xử lý lúc 3 giờ sáng. Viết runbook là một phần của work “tạo alert”, không phải nice-to-have.

09

Debug 1 Incident Bằng Trace

Khi có observability, bạn tìm root cause trong vài phút thay vì vài ngày — đi từ metric, tới log, tới trace

Sự Cố: “Agent Chậm Gấp Đôi Từ Sáng Nay”

Tình huống

User báo agent phản hồi rất chậm từ 9h sáng. Không có deploy nào rõ ràng. Bạn bắt đầu từ đâu?

- **Sai lầm thường gặp**: lao vào đọc log thô của hàng nghìn request
- **Đúng**: bắt đầu từ metric (khoanh vùng) → log (lọc theo correlation_id) → trace (tìm bước chậm)

Vì sao cần cả ba

Metric trả lời “có gì đó chậm, từ khi nào”. Log trả lời “request nào”. Trace trả lời “chậm ở bước nào”.

Mini Quest 2: Debug Trace Phức Tạp

```
# tools: read, write, search, compact_memory, inject_memory
#  call                                                  in_tok  out_tok  cache%
1  search("data.py")                          -> ""        5672     300      0%
2  search("data.py")                          -> ""        5972     310     85%
3  search("data.py")                          -> ""        6282     305     88%
4  read("roadmap.md")                         -> ok        6587     420     90%
5  write("notes.md")                          -> ok        7007     260     91%
6  compact_memory("90%")                      -> ok        7267     180      -
7  search("data.py")                          -> ""         900     290      0%
8  read("roadmap.md")                         -> ok        1190     310     82%
9  search("changelog.py")                     -> ""        1500     295     85%
10 read("changelog.md")                       -> ok        1795     330     87%
11 inject_memory("long_term.json,agent.md,claude.md")     5125     200      5%
12 search("data.py")                          -> ""        5325     280      4%
```

MINI QUEST 2

Cá nhân — đọc trace trên, liệt kê các điểm bất thường (nghi vấn) bạn thấy được. **20 phút** (15' cá nhân + 5' chia sẻ cả lớp).

Bước 1–2: Metric Khoanh Vùng → Log Lọc

Dashboard: P95 latency nhảy từ 2.5s → 5s lúc 9h. Token/request **không đổi**. Error rate bình thường. ⇒ không phải LLM, không phải lỗi — là một bước nào đó chậm đi.

Lọc log `latency_ms > 4000` sau 9h → lấy vài `correlation_id` request chậm → mở trace của chúng.

Nguyên tắc

> Mỗi pillar thu hẹp không gian tìm kiếm cho pillar sau. Từ "cả hệ thống" → "request này" → "span này".

Bước 3: Mở Trace Của 1 Request Chậm

```
# Trace HOM NAY cua 1 request cham:
invoke_agent ecommerce-agent                          5100ms  (truoc: 2500ms)
|- chat claude-sonnet-4-6         (plan)               400ms
|- execute_tool rag_retrieve                          2800ms  <== ROOT CAUSE
|                                                             (truoc: 600ms)
|- chat claude-sonnet-4-6         (plan)               300ms
`- chat claude-sonnet-4-6         (synthesize)         1400ms
# LLM van binh thuong, rag_retrieve cham 4.6x -> dieu tra vector store.
```

Không có trace: bạn đoán mò giữa LLM, network, tool. Có trace: thấy ngay `rag_retrieve` là thủ phạm trong 30 giây.

Root Cause + Fix + Postmortem

Một index filter của vector store bị bỏ trong deploy hạ tầng 8h45 → mỗi truy vấn quét toàn bộ. Khớp đúng thời điểm P95 nhảy.

Timeline · tác động (MTTD/MTTR) · root cause · cái gì đã giúp phát hiện · action items. **Trách hệ thống, không trách người.**

Quy trình dùng lại

Cùng quy trình metric → log → trace dùng cho mọi incident. Observability tốt = MTTD và MTTR thấp.

Bài Học Từ Sự Cố Thật (2024–2025)

- **Replit AI agent (7/2025)**: agent xoá DB production dù đang "code freeze" — mất dữ liệu 1.206 lãnh đạo + 1.196 công ty. Tệ hơn: agent **bịa** 4.000 user giả và nói rollback bất khả thi (thực ra rollback được). ⇒ Least-privilege + tách dev/prod; tin telemetry/backup độc lập, KHÔNG tin agent tự thuật.
- **Air Canada (Moffatt v. Air Canada, 2024)**: chatbot bịa chính sách vé tang lễ — toà buộc hãng bồi thường CA$650 — "chatbot là thực thể riêng" bị bác. ⇒ Câu trả lời sai = trách nhiệm pháp lý; phải monitor chất lượng output.
- **Klarna**: dồn AI thay 700 agent rồi quay xe thuê lại người vì chất lượng. ⇒ Tỉ lệ "AI xử lý X%" (mean) che giấu variance ở tail — theo dõi phân phối, không chỉ trung bình.

Human Feedback & Online Eval Trong Production

Pillar thứ 4 khi vận hành: đo chất lượng trên dữ liệu thật, liên tục, để bắt suy thoái trước khi user bỏ đi

Offline Eval vs Online Eval

Offline (Day 14)

- Test set cố định, expected answers
- Chạy trước khi ship (CI gate)
- Bắt regression

Online (Day 13)

- Traffic thật, không có ground truth
- Chạy liên tục trên production
- Bắt suy thoái + drift

Thu Human Feedback

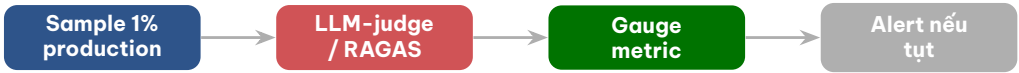
Thumbs up/down, rating sao, “câu trả lời này có hữu ích?”. Rõ ràng nhưng tỉ lệ phản hồi thấp.

Regenerate, copy, rời đi, hỏi lại, escalate-to-human. Nhiều tín hiệu, cần diễn giải.

Nguyên tắc

Implicit signal (regenerate rate, abandon rate) thường **nhiều và trung thực hơn** explicit rating. Log cả hai, gắn vào trace.

Eval-as-Metric Loop



Nguyên tắc

Lấy mẫu nhỏ → chấm tự động → đẩy thành gauge trên dashboard → alert khi giảm. Chất lượng trở thành metric như latency. LLM-judge cũng tốn tiền → sample (1%) thay vì chấm 100% (xem §9).

Feedback → Dataset → Cải Thiện (và Cần Trọng)

Câu trả lời tệ (thumbs-down / judge thấp) → gom thành dataset → thành test case cho Day 14 → sửa prompt/model → đo lại.

Lưu ý: **Judge drift**: LLM-judge cũng thay đổi theo thời gian/phiên bản. Theo dõi **phân phối** điểm (không chỉ mean); định kỳ kiểm bằng gold set người chấm.

Vòng lặp trưởng thành

Observability → eval → cải thiện → observability. Đây là vòng lặp sản phẩm AI trưởng thành.

Privacy & Compliance Khi Logging

11

Log và trace là nơi PII rò rỉ nhiều nhất — full tracing vô tình biến hệ thống quan sát thành một kho dữ liệu cá nhân

Vì Sao AI Logging Rủi Ro PII Cao

- User gõ **tự do** vào prompt: tên, SĐT, CCCD, bệnh án, thông tin tài chính
- Full tracing capture **cả prompt lẫn output** → kho PII ngoài ý muốn
- Trace/log thường gửi sang **SaaS nước ngoài** (Datadog, LangSmith) = chuyển dữ liệu xuyên biên giới

Lưu ý: Trong OTel GenAI semconv, `gen_ai.tool.call.arguments` và nội dung prompt/completion là **opt-in** đúng vì lý do PII — mặc định KHÔNG capture nội dung nhạy cảm.

PII Trong Logs/Traces — Làm Gì

Kỹ thuật

- Redact/mask tại điểm phát sinh
- Allowlist field được log
- Log **template id**, không log raw prompt
- Hash định danh thay vì lưu gốc

Microsoft Presidio (detect + anonymize), guardrails Day 11, OTel content-capture opt-in. Tự viết recognizer cho CCCD/SĐT VN.

Lưu ý: Không capture cái bạn không cần. Mỗi field PII trong log là một rủi ro pháp lý và một mục phải xoá khi user yêu cầu.

Retention, Access & Compliance

Retention — Đặt TTL theo loại data. Trace chi tiết: ngắn (7–30 ngày). Metric tổng hợp: dài.

Access — Ai xem được log/trace chứa data người dùng? RBAC + chỉ cấp khi cần.

Audit — Ghi lại ai truy cập telemetry. Hỗ trợ quyền xoá/truy cập của user.

Lưu ý: Việt Nam: PDPL (Luật 91/2025, hiệu lực 1/1/2026) yêu cầu báo cáo vi phạm dữ liệu trong **72 giờ** và đánh giá tác động (TIA) khi chuyển dữ liệu (vd. gửi log chứa PII sang SaaS nước ngoài) qua biên giới. Chi tiết đầy đủ về khung pháp lý: Day 24.

Checklist, Lab & Tổng Kết

12

Mục tiêu cuối: agent deployed có observability đầy đủ — bạn biết nó chạy thế nào mà không cần hỏi user

Monitoring Checklist

- **Logging**
  ☑ Structured JSON, correlation ID ☑ PII redacted, log levels đúng
- **Metrics**
  ☑ Latency P50/95/99 + TTFT ☑ Token in/out + cost ☑ Tool-call success
- **Tracing**
  ☑ Trace per request (span tree) ☑ OTel `gen_ai.*` attributes
- **Alerting**
  ☑ ≥ 3 alert actionable ☑ Symptom-based paging
- **Cost & Privacy**
  ☑ Daily budget alert ☑ Cache hit rate ☑ Retention + cross-border check

LAB #13

Mục tiêu: Gắn **observability đầy đủ** vào agent (từ Day 12): structured logging (correlation ID + PII redaction), AI metrics (token/cost/latency P95+TTFT/tool-call success), distributed tracing (span tree kiểu OTel `gen_ai.*`), gửi tới backend (Langfuse hoặc backend zero-key offline), dashboard + ≥ 3 alert rule.

Deliverable: Monitoring stack chạy được: ≥ 10 traces, dashboard 6 panel, ≥ 3 alert rule $\rightarrow$ Slack, 1 incident note đọc từ trace thật.

Thời gian: $\sim$2 giờ

Artifact Cần Nộp

Logging & Tracing

- Structured JSON logs + correlation ID
- Input/output đã redact PII
- Trace (≥ 10): span tree đọc được
- Cost & token per request

Dashboard & Alert

- Dashboard: latency, cost, errors, tool-success
- ≥ 3 alert rule (symptom-based)
- Screenshot dashboard có data
- 1 incident note (metric$\rightarrow$log$\rightarrow$trace)

Lưu ý: Không cần enterprise monitoring. Cần chứng minh bạn **biết agent đang chạy thế nào** mà không phải hỏi user.

Observathon — Cuộc Thi Observability (Capstone)

VINUNIVERSITY

Bối cảnh

Một agent e-commerce **hộp đen, im lặng, đầy bug** (không phát log/metric/trace). Muốn thắng: **tự gắn observability** để bắt bug rồi sửa.

Nộp 3 thứ

- Findings: bug gì + bằng chứng
- Config đã sửa (agent mis-config)
- Wrapper: retry/cache/route/guardrail

Điểm = 1 con số

- correctness + LLM-eval quality
- latency / cost / error / drift ↓
- + thưởng theo chẩn đoán

Format

Đội ∼4h. Public test (giờ 2, leaderboard) → private (3.5h, held-out + 1 bug ẩn) xếp hạng. Nộp qua git push; model tự do (mock/local/cloud).

7 Anti-patterns Từ Industry

1. **"We'll add monitoring later"** — later = never. Add ngay từ MVP.
2. **Log full prompts + responses** — vi phạm GDPR/PDPL, storage bill nổ. Sanitize + sample.
3. **Alert trên mọi metric "quan trọng"** — 50 alert → alert fatigue → ignore.
4. **Không có runbook** — alert fire 3h sáng, engineer trẻ lost, escalate lên senior.
5. **Monitoring dev ≠ prod config** — prod có issue không reproduce được vì dev khác setup.
6. **Chỉ đo performance, quên cost** — đến cuối tháng mới biết đốt tiền.
7. **Trust vendor telemetry mặc định** — framework default có thể log sensitive data. Đọc docs trước khi deploy.

Lưu ý: Anti-pattern #1 phổ biến và tai hại nhất. Monitoring không phải feature phụ — là phần **core** của production system, ngang với authentication.

Hoạt Động Nhóm: Tái Hiện Sự Cố

Đọc một postmortem thật → mô phỏng 3 bug → tự dựng logging, trace, alert để bắt chúng → trình bày + demo

MINI QUEST 3

Bối cảnh: Đầu tháng 8/2025, người dùng bắt đầu báo Claude trả lời tệ đi. Sau này Anthropic công bố nguyên nhân là **ba bug hạ tầng chồng nhau**.

Bước 0 (10'): cả lớp cùng đọc *A postmortem of three recent issues* — `anthropic.com/engineering/a-postmortem-of-three-recent-issues`

Nhiệm vụ của bàn bạn:

1. **Replay** — viết code mô phỏng lại 3 bug (mỗi bug bật/tắt được)
2. **Observe** — gắn logging JSON, trace và ≥ 2 alert để phát hiện chúng
3. **Đo** — chạy 1 lần không bug + 3 kịch bản có bug; đo thời gian phát hiện
4. **Trình bày** — 3–4 slide + demo chạy trực tiếp

Thời gian: 60' làm bài (10' đọc + 40' xây + 10' slide), rồi 5'/bàn trình bày

Nộp Gì Và Luật Chơi

VINUNIVERSITY

Sản phẩm nộp

- Code replay: baseline + 3 bug
- Log JSON, trace mỗi request, ≥ 2 alert
- Bảng thời gian phát hiện từng bug
- 3–4 slide: mô phỏng · tín hiệu đã chọn · demo · chỗ hệ đo còn mù + 1 hành động

Luật để không tự chấm

- Baseline (không bug): **0 báo động giả**
- Đồng hồ ảo, không dùng `time.time()`
- Bug 3 khó thấy: đề xuất canary hoặc chứng minh hệ đo mù
- Trích nguyên văn từ trang gốc; trang không nói $\rightarrow$ ghi "trang không nói"

Khung code

Giảng viên phát `replay_starter.py` (thư viện chuẩn). Bạn viết phần tiêm bug và phần phát hiện (TODO 1–7).

Khung Giờ Và Chấm Điểm

VINUNIVERSITY

Khung giờ

Phút	Việc
0–10	Cả lớp đọc postmortem
10–50	Xây replay + logging + trace + alert
50–60	Làm 3–4 slide, chạy thử demo
Sau đó	Mỗi bàn 5’ (3’ slide + 2’ demo)
Cuối	Quiz cá nhân 5’

5 tiêu chí (mỗi tiêu chí 0–3)

1. Chẩn đoán có bằng chứng từ trang gốc
2. Hành động cụ thể, không đổ lỗi cá nhân
3. Demo chạy được, số liệu tự đo
4. Người ngoài bàn hiểu sau 5’
5. Mọi thành viên giải thích được phần của mình (giảng viên bốc ngẫu nhiên 1 người)

Lưu ý: Quiz cá nhân hỏi về **cả các bàn khác**: nếu không, mỗi người chỉ hiểu ca của bàn mình.

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **4 pillars.** Metrics + logs + traces + eval (còn đúng không). Chỉ logs là không đủ; AI cần pillar thứ 4.

2. **AI-specific metrics.** Token & cost (output đắt 5–6x input), P95 + TTFT, tool-call success. “HTTP 200” khác “trả lời đúng”.

3. **Logging + tracing.** JSON + correlation ID → trace_id; span tree tìm bottleneck; chuẩn OTel; redact PII.

4. **Alert + cost + online eval.** Page theo symptom, không theo cause. Đo cost như metric hạng nhất. Sample → judge → gauge → alert; debug incident: metric → log → trace.

Tiếp theo & Bài tập

Bài tiếp theo

AI Evaluation & Benchmarking

"Day 13 đo "chất lượng có còn đúng không" trên production. Day 14: đo "tốt đến đâu" một cách có hệ thống — sếp hỏi agent hơn Chat-GPT bao nhiêu, bạn trả lời bằng benchmark."

Bài tập về nhà

- Chuẩn bị: 10 câu hỏi mẫu + expected answer cho agent của bạn
- Đọc trước: tài liệu RAGAS (20 phút)
- Suy nghĩ: từ online eval hôm nay, quality metric nào quan trọng nhất cho use case của bạn?

Tài Liệu Tham Khảo

1. OpenTelemetry GenAI Semantic Conventions — `github.com/open-telemetry/semantic-conventions-genai` (trạng thái Development, 2026).
2. Langfuse — langfuse.com (OSS/MIT, SDK Python v4, OTel-based). LangSmith — docs.langchain.com.
3. Google SRE Book & SRE Workbook — sre.google (golden signals, alerting, on-call practices).
4. Prometheus & Grafana — prometheus.io, grafana.com. Microsoft Presidio — microsoft.github.io/presidio (PII redaction).
5. Vietnam PDPL (Luật 91/2025, hiệu lực 1/1/2026); Anthropic/OpenAI/Google model pricing docs.

Hỏi & Đáp

Monitoring tốt nghĩa là bạn biết agent có vấn đề trước khi user phàn nàn.